# Contents

## Executive Summary

For this lab we learned about SNORT while primarily utilizing it on a Windows VM. The first section focused on the configuration of SNORT such as identifying the correct network interface and then ensuring it was correct. Once SNORT was set up properly, it was tested against traffic from a Kali machine using ping, netcat, and mnap. This traffic was captured in binary format and analyzed using Wireshark.

Once that was complete, we took a closer look at SNORT’s rules and syntax structure. To do so, we picked two CVEs and wrote our own rules for SNORT to use against them. This process helped build a stronger understanding of SNORT and how it could be utilized in a real-world scenario.

## Lab 4: Intrusion Detection

In this lab we are investigating SNORT and it's uses as an IDS software primarily on a Windows machine. First, we will start the software, then we will examine the configuration options. Next, we will view the traffic in WireShark. Finally, we will write our own custom SNORT rules to try on our own. "Snort is the foremost Open Source Intrusion Prevention System (IPS) in the world. Snort IPS uses a series of rules that help define malicious network activity and uses those rules to find packets that match against them and generates alerts for users" (Snort). Being an open-source software means that it is available to anyone for free, which can be helpful for the every day user or a small business who is unable to pay large fees for an IDS. As long as SNORT is configured properly, it can be a very effective tool in the cybersecurity world. Not only that, but it can be used as the basis to build one's own IPS software, although that does require a bit more expertise.

To start off, determining the correct network interface SNORT will use is important because SNORT needs to be able to capture the correct traffic in order to be effective. When SNORT is on the wrong interface, it won't be able to detect the threats it's supposed to, rendering it pointless. Setting SNORT up on the right interface is the first step to protection and getting valuable information.

Figure 1 shows an image of the network interface configuration. In order to see this, the command is "snort -W" which lists the available interfaces on Windows machines. The Index is the ID number used to communicate with SNORT. The Physical Address section shows the MAC address of the device. The IP address section shows the IP address of the device. The Device Name section shows the name of the device.

Lastly, the description is a human readable device name. When SNORT is run in-line, the network configuration will look the same.

In Figure 2, I tested and validated the configuration file by using the command “snort -T -c C:\Users\Adminstrator\Desktop\Snort\etc\snort.conf -i 3”. The -T flag tests and reports on the current SNORT configuration. The -c flag specifies the rules file. The -i flag tells SNORT what interface to listen on. I picked interface 3 because that is the machine that I am using with the IP address: 172.16.3.18. Interface 2 in this scenario was the control provided by SimSpace and would not have rendered useful when running SNORT.

Figure 3 shows SNORT having been started and processing packets. In order to start it, I used the command “snort -A console -i 3 -c C:\Users\Administrator\Desktop\Snort\etc\snort.conf -l (L) C:Users\Administrator\Desktop\Snort\log”. The -A flag tells SNORT what alert mode to run out of several options: fast, full, console, test, or none. For this instance, I just picked console because I wasn’t sure what I needed yet or how SNORT needed to be run for later in the lab. Next, the -l (L) flag tells SNORT to log its findings to a specific directory which can be found once SNORT is finished running.

The configuration files for SNORT are found in the \etc directory under SNORT. There are lots of ways to customize SNORT and make it work the way you want it to. The .conf file shown in Figure 4 has several samples to look at to get a better idea of how these files work. Nine examples are listed which are: set the network variables, configure the decoder, configure the base detection engine, configure dynamic loaded libraries, configure preprocessors, configure output plugins, customize your rule set, customize preprocessor and decoder rule set, customize shared object rule set.

Setting the network variables defines what network SNORT will be protecting. This is important because without the correct networks identified, SNORT could miss attacks. Configuring the decoder prepares raw data for inspection. This process can detect malicious packets early in the process. Configuring the base detection engine tells SNORT how to match packets to rules. This can affect performance and accuracy. Configuring Dynamic loaded libraries loads the dynamic modules to get them ready for detection. Configuring preprocessors gets them set up to analyze packets before they reach the detection engine. Protocols are analyzed before the rules are applied to detect specific types of attacks. Configuring output plugins tells SNORT where to send the alerts and logs it gets. This promotes organization and can help with integration later on. Customizing the rule set selects which rules SNORT will use. This will tailor detection for a specific environment and make SNORT more accurate. Customizing the preprocessor and decoder rule set are more specific rules about alerts. This helps single out the more severe alerts that could be lost in a lot of lower priority ones. Customizing shared object rule sets gets the advanced rules ready. These shared object rules are powerful detection tools that will help detection.

Figure 5 shows the rules included in the .conf file. Below these rules are the preprocessor and decoder rule set and the shared object rule set. All three of these sections define rules for SNORT. By looking at the rules that are not commented out in the .conf file, I determined that SNORT is configured for IDS mode currently. This is because when you view the rule file, the rules start with “alert”. This indicates IDS instead of IPS. Rules geared towards IPS would start with words like “block”, “reject”, or “drop”. An example of the app-detect rule is shown in Figure 6.

In order to change the location of the rules, I would add the line “var RULE_PATH filepath”. This tells SNORT where to find the rules within the directory. Then down in the snort.conf file I would write “include $RULE_PATH/example.rules”. This tells snort the rule to call on when it runs. In order to change SNORT to IPS, the rules would have to be changed from “alert” to “block”, “reject”, or “drop”. Then when running SNORT, it would need to be started in inline mode instead of IDS.

For this portion of the lab we had to run SNORT on the Windows VM while sending traffic to it from another. Then, we had to take the log we generated and put it into WireShark to analyze. Starting off with this section, I had to figure out the best way to run SNORT and I determined that running it without any rules would be fine for this test so I could see all traffic. The command I put into the terminal was “snort -i 3 -b -l C:\Users\Administrator\Desktop\Snort\log”. The -b flag told SNORT to make it a binary file, which would make it usable in WireShark. Once SNORT was running, I used my Kali VM (14 for reference) to send traffic. I started by pinging my machine using “ping 172.16.3.18”. Then I used the command “nc 172.16.3.18 80”. Lastly, I used “nmap -sS 172.16.3.18”. Between these three, it was enough to generate some packets for WireShark, shown in Figure 7.

Starting off, the first several packets (12-34) are the pings that I sent from the Kali machine (IP: 172.16.3.113) to the Windows machine (IP: 172.16.3.18). Pings are not an odd thing to see in a Wireshark capture and could be perfectly normal. Some normal reason to see a ping would be to check round-trip time or to make sure that the host is available. Though when they aren’t normal is when they occur in a high volume, such as an ICMP flood. “An Internet Control Message Protocol (ICMP) flood DDoS attack, also known as a Ping flood attack, is a common Denial-of-Service (DoS) attack in which an attacker attempts to overwhelm a targeted device with ICMP echo-requests (pings)” (NetScout). It is also abnormal to see ICMP echo requests that go out to multiple hosts or sequential IP addresses from one source. This behavior could indicate the start of an attack.

Next, I used netcat and attempted to connect to Port 80 on the Windows machine (packet 38). When I ran this, nothing ever popped up so I can only assume it was unsuccessful – or perhaps I didn’t wait long enough. Either way, the only thing that occurred in Wireshark was a TCP packet from the Kali machine to the Windows machine. Typically, there would be a TCP three-way handshake, though the handshake ended up never occurring. “Port 80 is the default network port for web servers using HTTP. It operates on the application layer of the TCP/IP networking model and serves as the communication gateway for HTTP requests and responses between client computers and servers” (NordVPN). Seeing an attempt to connect to Port 80 could be a major red flag and it’s important to monitor what happens next as it could indicate an attack, especially depending on where the request came from.

Lastly, I used nmap to attempt a TCP SYN scan which is a half-open scan, shown in Figure 8. “This technique is often referred to as half-open scanning, because you don't open a full TCP connection. You send a SYN packet, as if you are going to open a real connection and then wait for a response. A SYN/ACK indicates the port is listening (open), while a RST (reset) is indicative of a non-listener. If no response is received after several retransmissions, the port is marked as filtered" (Lyon). From packet 63 all the way to 2078, this scan occurred. This would be a major red flag as it's a good way for an attacker to tell what ports are open and potentially unprotected. To a defense team, it could be potentially indicative of someone preparing to enter a system. It would be important to see which ports were reported back as open so action could be taken quickly.

Overall, all of these alone could potentially be normal traffic, depending on the situation. However, it is important to note them and look for other indicators of signs of intrusion. All of these together, would certainly be concerning in a real scenario. Wireshark is such an essential tool when investigating happenings like this.

Snort is a very flexible tool that allows for users to write their own rules as needed. "While Snort rules are usually written in a single line, recent versions of Snort allow for multi-line rules; this is especially useful for more sophisticated rules that can be difficult to restrict to just one line. Snort rules consist of two logical parts: a rule header and rule options" (Lenaerts-Bergmans). Lenaerts-Bergmans goes on to say that header consists of an action to take, IP addresses, port numbers, direction of traffic, and inspection protocol. The rule options consist of a message to display when the rule matches, flow state, content or pattern, service or application protocol, and Snort ID and revision number. These components are important to remember when writing custom rules because otherwise, they likely will not work.

A CVE stands for common vulnerabilities and exposures. To pick two CVEs, I went to cisa.gov and reviewed the list that they keep. For my first CVE I picked CVE-2025-24054 named the Microsoft Windows NTLM Hash Disclosure Spoofing

Vulnerability. This vulnerability is described as “Microsoft Windows NTLM contains an external control of file name or path vulnerability that allows an unauthorized attacker to perform spoofing over a network” (CISA). I am not extremely knowledgeable on SNORT rules and did need a little assistance from ChatGPT. The rule I wrote was: “alert tcp $HOME_NET any -> !$HOME_NET 445 (msg:"Possible CVE-2025-24054 NTLM hash leak via SMB"; flow:established,to_server; content:"NTLMSSP"; offset:0; depth:100; sid:1000001; rev:1;)”. This rule alerts on any outbound TCP traffic on port 445, which is used for SMB. The flow applies to established connections directed to a server. The content portion is looking for NTLM authentication attempts. The offset and depth is looking at the first 100 bytes of the packet where the signature for NTLMSSP would be. Lastly, a SNORT ID and revision ID is assigned. This rule would not definitively identify this CVE but it would be a step in the right direction and more investigation would need to be done to confirm. Figure 9 shows the rules file.

My next CVE I chose is CVE-2025-31161 named CrushFTP Authentication Bypass Vulnerability. This is defined as “CrushFTP contains an authentication bypass vulnerability in the HTTP authorization header that allows a remote unauthenticated attacker to authenticate to any known or guessable user account (e.g., crushadmin), potentially leading to a full compromise” (CISA) The rule I wrote was “alert tcp $EXTERNAL_NET any -> $HOME_NET [80,443] (msg:"CVE-2025-31161 CrushFTP Authentication Bypass Attempt"; flow:to_server,established; content:"Authorization: AWS4-HMAC-SHA256 Credential="; nocase; http_header; content:"/"; within:50; http_header; content:"Cookie: CrushAuth="; nocase; http_header; classtype:attempted-admin; sid:1000003; rev:1;)”. This rule is an alert that alerts for any TCP traffic coming from an external source on port 80 and 443 which are for HTTP and HTTPS, respectively. The flow is the same as above which is established connections. The content detects the specific authentication header used and the http header portion states that the content portion should be found in the header. The forward slash should follow the "Credential=" within 50 bytes. The next content portion is checking for the CrushAuth cookie which is also part of the exploit. The class type is an attempted administrative access. This rule covers most of the bases but could potentially cause false positives or may miss some attempts in this vulnerability. The rule is shown in Figure 10 and Figure 11 shows SNORT running successfully with both rules in place.

This lab was a good opportunity to learn about some basics involving intrusion detection. Not only that but it provided information over the tool SNORT which has lots of uses and can assist when intrusions occur in real time. Overall, SNORT itself was relatively easy to understand but the rules themselves were not the easiest things to write.

## Bibliography

CISA. "Known Exploited Vulnerabilities Catalog | CISA." *Www.cisa.gov*, www.cisa.gov/known-exploited-vulnerabilities-catalog?page=1.

Lenaerts-Bergmans, Bart. "Snort Explained: Understanding Snort Rules and Use Cases | CrowdStrike." *Crowdstrike.com*, 2024, www.crowdstrike.com/en-us/cybersecurity-101/threat-intelligence/snort-rules/.

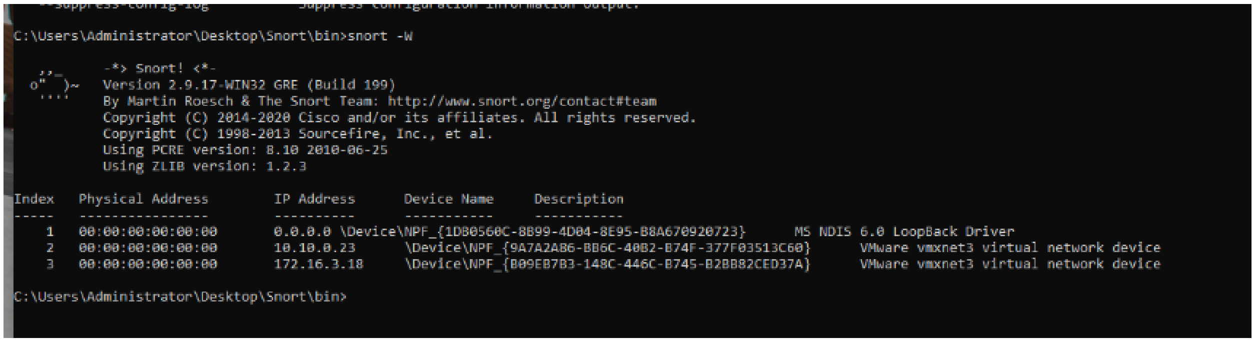
Lyon, Gordon. "Port Scanning Techniques | Nmap Network Scanning." *Nmap.org*, 2019, nmap.org/book/man-port-scanning-techniques.html.

NetScout. "What Is an ICMP Flood Attack?" *NETSCOUT*, 2025, www.netscout.com/what-is-ddos/icmp-flood?utm_source=chatgpt.com. Accessed 14 May 2025.

NordVPN. "Port 80 Definition – Glossary | NordVPN." *Nordvpn.com*, 22 Aug. 2023, nordvpn.com/cybersecurity/glossary/port-80/.

Snort. "Snort - Network Intrusion Detection & Prevention System." *Snort.org*, Snort, 2019, www.snort.org/.

# Appendix



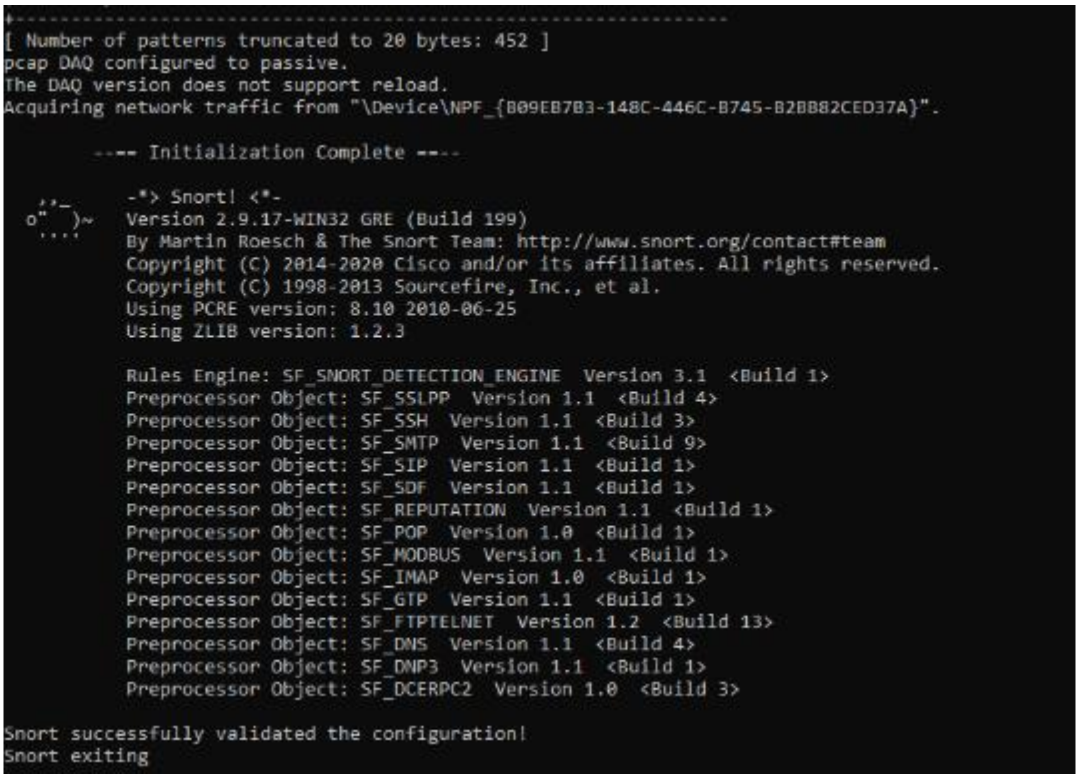
```
C:\Users\Administrator\Desktop\Snort\bin>snort -W

   ,,_     -*> Snort! <*-
  o"  )~   Version 2.9.17-WIN32 GRE (Build 199)
   ''''    By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
           Copyright (C) 2014-2020 Cisco and/or its affiliates. All rights reserved.
           Copyright (C) 1998-2013 Sourcefire, Inc., et al.
           Using PCRE version: 8.10 2010-06-25
           Using ZLIB version: 1.2.3

Index   Physical Address        IP Address      Device Name     Description
-----   ----------------        ----------      -----------     -----------
    1   00:00:00:00:00:00       0.0.0.0 \Device\NPF_{1DB0560C-8B99-4D04-8E95-B8A670920723}      MS NDIS 6.0 LoopBack Driver
    2   00:00:00:00:00:00       10.10.0.23      \Device\NPF_{9A7A2AB6-BB6C-40B2-B74F-377F03513C60}      VMware vmxnet3 virtual network device
    3   00:00:00:00:00:00       172.16.3.18     \Device\NPF_{B09EB7B3-148C-446C-B745-B2BB82CED37A}      VMware vmxnet3 virtual network device

C:\Users\Administrator\Desktop\Snort\bin>
```

*Figure 1: Network interface configuration*



```
[ Number of patterns truncated to 20 bytes: 452 ]
pcap DAQ configured to passive.
The DAQ version does not support reload.
Acquiring network traffic from "\Device\NPF_{B09EB7B3-148C-446C-B745-B2BB82CED37A}".

        --== Initialization Complete ==--

   ,,_     -*> Snort! <*-
  o"  )~   Version 2.9.17-WIN32 GRE (Build 199)
   ''''    By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
           Copyright (C) 2014-2020 Cisco and/or its affiliates. All rights reserved.
           Copyright (C) 1998-2013 Sourcefire, Inc., et al.
           Using PCRE version: 8.10 2010-06-25
           Using ZLIB version: 1.2.3

           Rules Engine: SF_SNORT_DETECTION_ENGINE  Version 3.1  <Build 1>
           Preprocessor Object: SF_SSLPP  Version 1.1  <Build 4>
           Preprocessor Object: SF_SSH  Version 1.1  <Build 3>
           Preprocessor Object: SF_SMTP  Version 1.1  <Build 9>
           Preprocessor Object: SF_SIP  Version 1.1  <Build 1>
           Preprocessor Object: SF_SDF  Version 1.1  <Build 1>
           Preprocessor Object: SF_REPUTATION  Version 1.1  <Build 1>
           Preprocessor Object: SF_POP  Version 1.0  <Build 1>
           Preprocessor Object: SF_MODBUS  Version 1.1  <Build 1>
           Preprocessor Object: SF_IMAP  Version 1.0  <Build 1>
           Preprocessor Object: SF_GTP  Version 1.1  <Build 1>
           Preprocessor Object: SF_FTPTELNET  Version 1.2  <Build 13>
           Preprocessor Object: SF_DNS  Version 1.1  <Build 4>
           Preprocessor Object: SF_DNP3  Version 1.1  <Build 1>
           Preprocessor Object: SF_DCERPC2  Version 1.0  <Build 3>

Snort successfully validated the configuration!
Snort exiting
```

*Figure 2: Tested and validated configuration file*

```
[ Number of patterns truncated to 20 bytes: 452 ]
pcap DAQ configured to passive.
The DAQ version does not support reload.
Acquiring network traffic from "\Device\NPF_{B09EB7B3-148C-446C-B745-B2BB82CED37A}".
Decoding Ethernet

        --== Initialization Complete ==--

   ,,_     -*> Snort! <*-
  o"  )~   Version 2.9.17-WIN32 GRE (Build 199)
   ''''    By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
           Copyright (C) 2014-2020 Cisco and/or its affiliates. All rights reserved.
           Copyright (C) 1998-2013 Sourcefire, Inc., et al.
           Using PCRE version: 8.10 2010-06-25
           Using ZLIB version: 1.2.3

           Rules Engine: SF_SNORT_DETECTION_ENGINE  Version 3.1  <Build 1>
           Preprocessor Object: SF_SSLPP  Version 1.1  <Build 4>
           Preprocessor Object: SF_SSH  Version 1.1  <Build 3>
           Preprocessor Object: SF_SMTP  Version 1.1  <Build 9>
           Preprocessor Object: SF_SIP  Version 1.1  <Build 1>
           Preprocessor Object: SF_SDF  Version 1.1  <Build 1>
           Preprocessor Object: SF_REPUTATION  Version 1.1  <Build 1>
           Preprocessor Object: SF_POP  Version 1.0  <Build 1>
           Preprocessor Object: SF_MODBUS  Version 1.1  <Build 1>
           Preprocessor Object: SF_IMAP  Version 1.0  <Build 1>
           Preprocessor Object: SF_GTP  Version 1.1  <Build 1>
           Preprocessor Object: SF_FTPTELNET  Version 1.2  <Build 13>
           Preprocessor Object: SF_DNS  Version 1.1  <Build 4>
           Preprocessor Object: SF_DNP3  Version 1.1  <Build 1>
           Preprocessor Object: SF_DCERPC2  Version 1.0  <Build 3>
Commencing packet processing (pid=4128)
```

*Figure 3: SNORT started*

```
File  Edit  Format  View  Help
#     This configuration file enables active response, to run snort in
#     test mode -T you are required to supply an interface -i <interface>
#     or test mode will fail to fully validate the configuration and
#     exit with a FATAL error
#--------------------------------------------------

###################################################
# This file contains a sample snort configuration.
# You should take the following steps to create your own custom configuration:
#
#  1) Set the network variables.
#  2) Configure the decoder
#  3) Configure the base detection engine
#  4) Configure dynamic loaded libraries
#  5) Configure preprocessors
#  6) Configure output plugins
#  7) Customize your rule set
#  8) Customize preprocessor and decoder rule set
#  9) Customize shared object rule set
###################################################

###################################################
# Step #1: Set the network variables.  For more information, see README.variables
###################################################

# Setup the network addresses you are protecting
ipvar HOME_NET 172.16.3.0/24

# Set up the external network addresses. Leave as "any" in most situations
ipvar EXTERNAL_NET any

# List of DNS servers on your network
ipvar DNS_SERVERS $HOME_NET

# List of SMTP servers on your network
ipvar SMTP_SERVERS $HOME_NET

# List of web servers on your network
ipvar HTTP_SERVERS $HOME_NET

# List of sql servers on your network
ipvar SQL_SERVERS $HOME_NET

# List of telnet servers on your network
ipvar TELNET_SERVERS $HOME_NET

# List of ssh servers on your network
ipvar SSH_SERVERS $HOME_NET

# List of ftp servers on your network
ipvar FTP_SERVERS $HOME_NET

# List of sip servers on your network
ipvar SIP_SERVERS $HOME_NET

# List of ports you run web servers on
portvar HTTP_PORTS [80,81,311,383,591,593,901,1220,1414,1741,1830,2301,2381,2809,3037,3128,3702,4343,4848,5250,6988,7000,7001,7144,7145,7510,7777,7779,8000,8008,8014,8028,8080,8085,8088,8090,8118,8123,8180,8181,8243,8280,8300,8800,8888,8899,9000,9060,9080,9090,9091,9443,9999,11371,34443,34444,41080,50002,55555]

# List of ports you want to look for SHELLCODE on.
portvar SHELLCODE_PORTS !80

# List of ports you might see oracle attacks on
portvar ORACLE_PORTS 1024:

# List of ports you want to look for SSH connections on:
portvar SSH_PORTS 22

# List of ports you run ftp servers on
```

*Figure 4: .conf file*

```
#####################################################
# Step #7: Customize your rule set
# For more information, see Snort Manual, Writing !
#
# NOTE: All categories are enabled in this conf fi
#####################################################

# site specific rules
include $RULE_PATH\local.rules

include $RULE_PATH\app-detect.rules
include $RULE_PATH\attack-responses.rules
include $RULE_PATH\backdoor.rules
include $RULE_PATH\bad-traffic.rules
include $RULE_PATH\blacklist.rules
include $RULE_PATH\botnet-cnc.rules
include $RULE_PATH\browser-chrome.rules
include $RULE_PATH\browser-firefox.rules
include $RULE_PATH\browser-ie.rules
include $RULE_PATH\browser-other.rules
include $RULE_PATH\browser-plugins.rules
include $RULE_PATH\browser-webkit.rules
include $RULE_PATH\chat.rules
include $RULE_PATH\content-replace.rules
include $RULE_PATH\ddos.rules
include $RULE_PATH\dns.rules
include $RULE_PATH\dos.rules
include $RULE_PATH\experimental.rules
include $RULE_PATH\exploit-kit.rules
include $RULE_PATH\exploit.rules
include $RULE_PATH\file-executable.rules
include $RULE_PATH\file-flash.rules
include $RULE_PATH\file-identify.rules
include $RULE_PATH\file-image.rules
include $RULE_PATH\file-multimedia.rules
include $RULE_PATH\file-office.rules
include $RULE_PATH\file-other.rules
include $RULE_PATH\file-pdf.rules
include $RULE_PATH\finger.rules
include $RULE_PATH\ftp.rules
include $RULE_PATH\icmp-info.rules
include $RULE_PATH\icmp.rules
include $RULE_PATH\imap.rules
include $RULE_PATH\indicator-compromise.rules
include $RULE_PATH\indicator-obfuscation.rules
include $RULE_PATH\indicator-shellcode.rules
include $RULE_PATH\info.rules
include $RULE_PATH\malware-backdoor.rules
include $RULE_PATH\malware-cnc.rules
include $RULE_PATH\malware-other.rules
include $RULE_PATH\malware-tools.rules
include $RULE_PATH\misc.rules
include $RULE_PATH\multimedia.rules
include $RULE_PATH\mysql.rules
#include $RULE_PATH\netbios.rules
include $RULE_PATH\nntp.rules
include $RULE_PATH\oracle.rules
#include $RULE_PATH\os-linux.rules
#include $RULE_PATH\os-other.rules
#include $RULE_PATH\os-solaris.rules
#include $RULE_PATH\os-windows.rules
include $RULE_PATH\other-ids.rules
include $RULE_PATH\p2p.rules
include $RULE_PATH\phishing-spam.rules
include $RULE_PATH\policy-multimedia.rules
#include $RULE_PATH\policy-other.rules
include $RULE_PATH\policy.rules
include $RULE_PATH\policy-social.rules
include $RULE_PATH\policy-spam.rules
include $RULE_PATH\pop2.rules
include $RULE_PATH\pop3.rules
include $RULE_PATH\protocol-finger.rules
include $RULE_PATH\protocol-ftp.rules
#include $RULE_PATH\protocol-icmp.rules
#include $RULE_PATH\protocol-imap.rules
#include $RULE_PATH\protocol-pop.rules
include $RULE_PATH\protocol-services.rules
#include $RULE_PATH\protocol-voip.rules
#include $RULE_PATH\pua-adware.rules
#include $RULE_PATH\pua-other.rules
#include $RULE_PATH\pua-p2p.rules
#include $RULE_PATH\pua-toolbars.rules
include $RULE_PATH\rpc.rules
include $RULE_PATH\rservices.rules
include $RULE_PATH\scada.rules
include $RULE_PATH\scan.rules
#include $RULE_PATH\server-apache.rules
include $RULE_PATH\server-iis.rules
#include $RULE_PATH\server-mail.rules
#include $RULE_PATH\server-mssql.rules
#include $RULE_PATH\server-mysql.rules
#include $RULE_PATH\server-oracle.rules
#include $RULE_PATH\server-other.rules
#include $RULE_PATH\server-webapp.rules
include $RULE_PATH\shellcode.rules
include $RULE_PATH\smtp.rules
include $RULE_PATH\snmp.rules
include $RULE_PATH\specific-threats.rules
include $RULE_PATH\spyware-put.rules
include $RULE_PATH\sql.rules
include $RULE_PATH\telnet.rules
include $RULE_PATH\tftp.rules
include $RULE_PATH\virus.rules
include $RULE_PATH\voip.rules
include $RULE_PATH\web-activex.rules
include $RULE_PATH\web-attacks.rules
include $RULE_PATH\web-cgi.rules
include $RULE_PATH\web-client.rules
include $RULE_PATH\web-coldfusion.rules
include $RULE_PATH\web-frontpage.rules
include $RULE_PATH\web-iis.rules
include $RULE_PATH\web-misc.rules
include $RULE_PATH\web-php.rules
include $RULE_PATH\x11.rules
```

*Figure 5: Rules in SNORT.conf file*

app-detect - Notepad
File Edit Format View Help

```
# Copyright 2001-2021 Sourcefire, Inc. All Rights Reserved.
#
# This file contains (i) proprietary rules that were created, tested and certified by
# Sourcefire, Inc. (the "VRT Certified Rules") that are distributed under the VRT
# Certified Rules License Agreement (v 2.0), and (ii) rules that were created by
# Sourcefire and other third parties (the "GPL Rules") that are distributed under the
# GNU General Public License (GPL), v2.
#
# The VRT Certified Rules are owned by Sourcefire, Inc. The GPL Rules were created
# by Sourcefire and other third parties. The GPL Rules created by Sourcefire are
# owned by Sourcefire, Inc., and the GPL Rules not created by Sourcefire are owned by
# their respective creators. Please see http://www.snort.org/snort/snort-team/ for a
# list of third party owners and their respective copyrights.
#
# In order to determine what rules are VRT Certified Rules or GPL Rules, please refer
# to the VRT Certified Rules License Agreement (v2.0).
#
#------------------
# APP-DETECT RULES
#------------------

# alert tcp $HOME_NET any -> $EXTERNAL_NET [80,443,8200] (msg:"APP-DETECT GoToMyPC remote control att
# alert tcp $EXTERNAL_NET any -> $HOME_NET 3389 (msg:"APP-DETECT remote desktop protocol attempted ac
# alert udp any any -> 255.255.255.255 23945 (msg:"APP-DETECT Data Rescue IDA Pro startup license che
alert tcp $HOME_NET [2601:2606] -> $EXTERNAL_NET any (msg:"APP-DETECT Quagga password challenge detec
# alert tcp $HOME_NET any -> $EXTERNAL_NET $HTTP_PORTS (msg:"APP-DETECT HTTPTunnel proxy outbound cor
# alert udp any 68 -> any 67 (msg:"APP-DETECT Intel AMT DHCP boot request detected"; flow:to_server;
# alert tcp $EXTERNAL_NET any -> $HTTP_SERVERS $HTTP_PORTS (msg:"APP-DETECT OpenVAS Scanner User-Ager
# alert tcp $HOME_NET any -> $EXTERNAL_NET $HTTP_PORTS (msg:"APP-DETECT Bloomberg web crawler outbour
# alert tcp $EXTERNAL_NET any -> $HOME_NET $HTTP_PORTS (msg:"APP-DETECT Jenkins Groovy script access
# alert tcp $HOME_NET any -> $EXTERNAL_NET $HTTP_PORTS (msg:"APP-DETECT Hola VPN startup attempt"; fl
# alert tcp $HOME_NET any -> $EXTERNAL_NET [$HTTP_PORTS,22222,22223] (msg:"APP-DETECT Hola VPN tunnel
```

*Figure 6: app-detect rule*

| | | | | | | |
|---|---|---|---|---|---|---|
| 1 | 0.000000 | 172.16.3.18 | 172.16.2.3 | DNS | 86 | Standard query 0xccd0 A slscr.update.microsoft.com |
| 2 | 2.843267 | 172.16.2.3 | 172.16.3.18 | DNS | 86 | Standard query response 0x6bfc Server failure A slscr.update.microsoft.com |
| 3 | 4.016262 | 172.16.3.18 | 172.16.2.3 | DNS | 78 | Standard query 0x5ac8 A ISCS-FILE.site.lan |
| 4 | 4.017202 | 172.16.2.3 | 172.16.3.18 | DNS | 94 | Standard query response 0x5ac8 A ISCS-FILE.site.lan A 172.16.2.2 |
| 5 | 4.021633 | 172.16.3.18 | 172.16.2.3 | DNS | 86 | Standard query 0x5e8f A slscr.update.microsoft.com |
| 6 | 4.046898 | 172.16.3.18 | 172.16.2.3 | DNS | 86 | Standard query 0x5e8f A slscr.update.microsoft.com |
| 7 | 5.062404 | 172.16.3.18 | 172.16.2.3 | DNS | 86 | Standard query 0x5e8f A slscr.update.microsoft.com |
| 8 | 7.062553 | 172.16.3.18 | 172.16.2.3 | DNS | 86 | Standard query 0x5e8f A slscr.update.microsoft.com |
| 9 | 7.382131 | 172.16.2.3 | 172.16.3.18 | DNS | 86 | Standard query response 0xccd0 Server failure A slscr.update.microsoft.com |
| 10 | 7.382133 | 172.16.2.3 | 172.16.3.18 | DNS | 86 | Standard query response 0xccd0 Server failure A slscr.update.microsoft.com |
| 11 | 7.382259 | 172.16.2.3 | 172.16.3.18 | DNS | 86 | Standard query response 0xccd0 Server failure A slscr.update.microsoft.com |
| 12 | 7.925964 | 172.16.3.113 | 172.16.3.18 | ICMP | 98 | Echo (ping) request id=0x1297, seq=1/256, ttl=64 (reply in 13) |
| 13 | 7.926136 | 172.16.3.18 | 172.16.3.113 | ICMP | 98 | Echo (ping) reply id=0x1297, seq=1/256, ttl=128 (request in 12) |
| 14 | 8.931101 | 172.16.3.113 | 172.16.3.18 | ICMP | 98 | Echo (ping) request id=0x1297, seq=2/512, ttl=64 (reply in 15) |
| 15 | 8.931247 | 172.16.3.18 | 172.16.3.113 | ICMP | 98 | Echo (ping) reply id=0x1297, seq=2/512, ttl=128 (request in 14) |
| 16 | 9.955091 | 172.16.3.113 | 172.16.3.18 | ICMP | 98 | Echo (ping) request id=0x1297, seq=3/768, ttl=64 (reply in 17) |
| 17 | 9.955250 | 172.16.3.18 | 172.16.3.113 | ICMP | 98 | Echo (ping) reply id=0x1297, seq=3/768, ttl=128 (request in 16) |
| 18 | 10.979185 | 172.16.3.113 | 172.16.3.18 | ICMP | 98 | Echo (ping) request id=0x1297, seq=4/1024, ttl=64 (reply in 19) |
| 19 | 10.979337 | 172.16.3.18 | 172.16.3.113 | ICMP | 98 | Echo (ping) reply id=0x1297, seq=4/1024, ttl=128 (request in 18) |
| 20 | 11.007050 | 172.16.2.3 | 172.16.3.18 | DNS | 86 | Standard query response 0xccd0 Server failure A slscr.update.microsoft.com |
| 21 | 11.078076 | 172.16.3.18 | 172.16.2.3 | DNS | 86 | Standard query 0x5e8f A slscr.update.microsoft.com |
| 22 | 12.003237 | 172.16.3.113 | 172.16.3.18 | ICMP | 98 | Echo (ping) request id=0x1297, seq=5/1280, ttl=64 (reply in 23) |
| 23 | 12.003378 | 172.16.3.18 | 172.16.3.113 | ICMP | 98 | Echo (ping) reply id=0x1297, seq=5/1280, ttl=128 (request in 22) |
| 24 | 12.765420 | Intel_00:08:1b | Intel_00:07:1b | ARP | 42 | Who has 172.16.3.113? Tell 172.16.3.18 |
| 25 | 12.765738 | Intel_00:07:1b | Intel_00:08:1b | ARP | 60 | 172.16.3.113 is at 00:02:b3:00:07:1b |
| 26 | 12.995156 | Intel_00:07:1b | Intel_00:08:1b | ARP | 60 | Who has 172.16.3.18? Tell 172.16.3.113 |
| 27 | 12.995203 | Intel_00:08:1b | Intel_00:07:1b | ARP | 42 | 172.16.3.18 is at 00:02:b3:00:08:1b |
| 28 | 13.027446 | 172.16.3.113 | 172.16.3.18 | ICMP | 98 | Echo (ping) request id=0x1297, seq=6/1536, ttl=64 (reply in 29) |
| 29 | 13.027651 | 172.16.3.18 | 172.16.3.113 | ICMP | 98 | Echo (ping) reply id=0x1297, seq=6/1536, ttl=128 (request in 28) |
| 30 | 14.051258 | 172.16.3.113 | 172.16.3.18 | ICMP | 98 | Echo (ping) request id=0x1297, seq=7/1792, ttl=64 (reply in 31) |
| 31 | 14.051454 | 172.16.3.18 | 172.16.3.113 | ICMP | 98 | Echo (ping) reply id=0x1297, seq=7/1792, ttl=128 (request in 30) |
| 32 | 14.644452 | 172.16.2.3 | 172.16.3.18 | DNS | 86 | Standard query response 0xccd0 Server failure A slscr.update.microsoft.com |
| 33 | 15.075391 | 172.16.3.113 | 172.16.3.18 | ICMP | 98 | Echo (ping) request id=0x1297, seq=8/2048, ttl=64 (reply in 34) |
| 34 | 15.075617 | 172.16.3.18 | 172.16.3.113 | ICMP | 98 | Echo (ping) reply id=0x1297, seq=8/2048, ttl=128 (request in 33) |
| 35 | 15.088536 | 172.16.3.18 | 172.16.2.3 | DNS | 86 | Standard query 0x37b6 A slscr.update.microsoft.com |
| 36 | 15.109305 | 172.16.3.18 | 172.16.2.3 | DNS | 86 | Standard query 0x37b6 A slscr.update.microsoft.com |
| 37 | 16.124910 | 172.16.3.18 | 172.16.2.3 | DNS | 86 | Standard query 0x37b6 A slscr.update.microsoft.com |
| 38 | 17.768477 | 172.16.3.113 | 172.16.3.18 | TCP | 74 | 37834 → 80 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=4138807592 TSecr=0 WS=128 |
| 39 | 18.140526 | 172.16.3.18 | 172.16.2.3 | DNS | 86 | Standard query 0x37b6 A slscr.update.microsoft.com |
| 40 | 18.787447 | 172.16.3.113 | 172.16.3.18 | TCP | 74 | [TCP Retransmission] 37834 → 80 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=4138808611 TSecr=0 WS=128 |
| 41 | 19.194570 | 172.16.2.3 | 172.16.3.18 | DNS | 86 | Standard query response 0x5e8f Server failure A slscr.update.microsoft.com |
| 42 | 19.194571 | 172.16.2.3 | 172.16.3.18 | DNS | 86 | Standard query response 0x5e8f Server failure A slscr.update.microsoft.com |
| 43 | 19.194908 | 172.16.2.3 | 172.16.3.18 | DNS | 86 | Standard query response 0x5e8f Server failure A slscr.update.microsoft.com |
| 44 | 19.195502 | 172.16.2.3 | 172.16.3.18 | DNS | 86 | Standard query response 0x5e8f Server failure A slscr.update.microsoft.com |
| 45 | 19.765453 | Intel_00:08:1b | Intel_00:0c:01 | ARP | 42 | Who has 172.16.3.1? Tell 172.16.3.18 |
| 46 | 19.765826 | Intel_00:0c:01 | Intel_00:08:1b | ARP | 60 | 172.16.3.1 is at 00:02:b3:00:0c:01 |
| 47 | 20.803489 | 172.16.3.113 | 172.16.3.18 | TCP | 74 | [TCP Retransmission] 37834 → 80 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=4138810627 TSecr=0 WS=128 |

*Figure 7: WireShark from log*

| | | | | | | |
|---|---|---|---|---|---|---|
| 141 | 29.738407 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48763 → 1007 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 142 | 29.738422 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48763 → 1046 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 143 | 29.738422 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48763 → 3905 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 144 | 29.738424 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48763 → 1755 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 145 | 29.738426 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48763 → 700 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 146 | 29.740637 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48763 → 1121 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 147 | 29.740638 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48763 → 1556 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 148 | 29.740639 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48763 → 6001 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 149 | 29.740653 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48763 → 32768 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 150 | 29.742778 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48763 → 1328 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 151 | 29.742780 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48763 → 7496 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 152 | 29.742796 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48763 → 1580 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 153 | 29.742797 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48763 → 1812 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 154 | 29.742798 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48763 → 5925 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 155 | 29.742814 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48763 → 19283 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 156 | 29.742835 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48763 → 1147 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 157 | 29.742842 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48763 → 8031 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 158 | 29.742889 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48763 → 5120 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 159 | 29.742897 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48763 → 3737 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 160 | 29.838540 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48764 → 1007 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 161 | 29.838565 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48764 → 1755 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 162 | 29.838566 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48764 → 700 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 163 | 29.838567 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48764 → 1046 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 164 | 29.838596 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48764 → 4 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 165 | 29.838597 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48764 → 10010 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 166 | 29.838609 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48764 → 3905 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 167 | 29.840664 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48764 → 1556 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 168 | 29.840715 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48764 → 32768 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 169 | 29.840727 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48764 → 1121 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 170 | 29.840729 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48764 → 6001 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 171 | 29.842782 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48764 → 19283 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 172 | 29.842886 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48764 → 1812 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 173 | 29.842888 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48764 → 1580 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 174 | 29.842891 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48764 → 1147 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 175 | 29.842892 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48764 → 1328 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 176 | 29.842893 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48764 → 7496 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 177 | 29.844969 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48764 → 5925 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 178 | 29.844970 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48764 → 8031 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 179 | 29.844983 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48764 → 5120 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 180 | 29.844985 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48764 → 3737 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 181 | 29.938684 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48763 → 14441 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 182 | 29.938690 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48763 → 8085 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 183 | 29.938690 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48763 → 32771 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 184 | 29.938691 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48763 → 65000 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 185 | 29.938691 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48763 → 7999 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 186 | 29.938736 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48763 → 2968 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 187 | 29.938736 | 172.16.3.113 | 172.16.3.18 | TCP | 60 | 48763 → 6839 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |

*Figure 8: Half-open scan*

A.CustomRule1 - Notepad
File Edit Format View Help

```
alert tcp $HOME_NET any -> !$HOME_NET 445 (msg:"Possible CVE-2025-24054 NTLM hash leak via SMB"; flow:established,to_server; content:"NTLMSSP"; offset:0; depth:100; sid:1000001; rev:1;)
```

*Figure 9: The first custom rule*


*Figure 10: My second custom rule*

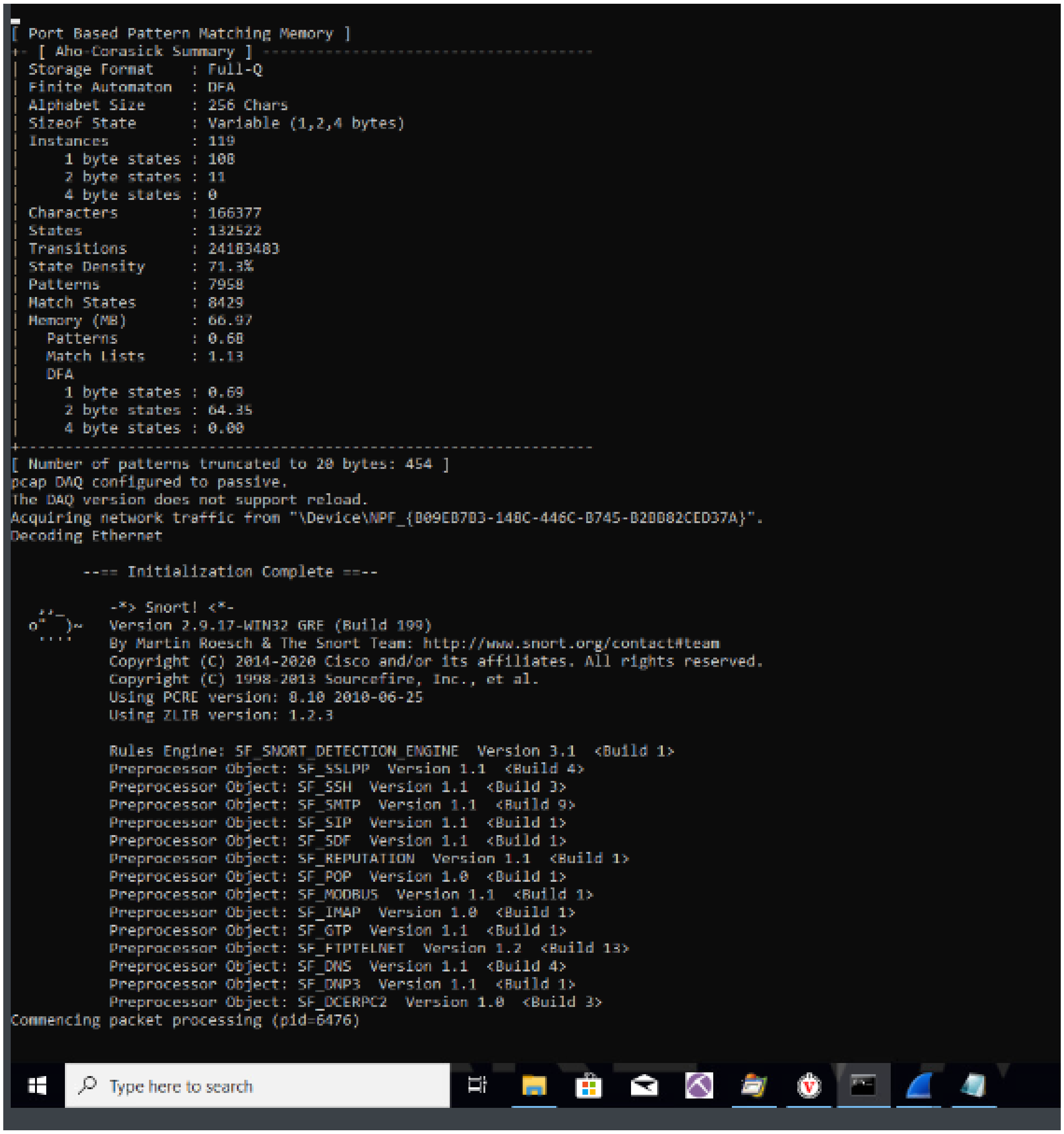



*Figure 11: SNORT running with both rules active*